

# “Vulnerable Bank – Security Assessment Report”

## 1. SCOPE OF ASSESSMENT

The assessment was carried out on the locally hosted 'Vulnerable Bank' web application. The evaluation focused on the following modules:

- User authentication and login process
- Balance and transaction management
- Password reset and recovery flow
- AI-based customer support chatbot
- Client-side price and payment handling features

## 2. METHODOLOGY

The testing methodology combined manual inspection and automated vulnerability scanning techniques. The process included deploying the application locally, analyzing network requests, attempting input manipulation, and executing simulated attacks to identify weak points. Tools like Burp Suite and browser developer consoles were used for testing SQL injection, input tampering, and prompt manipulation vulnerabilities.

## 3. VULNERABILITY FINDINGS

### 01. Authentication Bypass - SQL Injection

**Risk Level: HIGH**

**What:** Authentication Bypass through SQL Injection occurs when an attacker manipulates the SQL query used in the authentication process to bypass the need for valid credentials. This is typically achieved by injecting SQL code into the username or password fields to alter the query's logic.

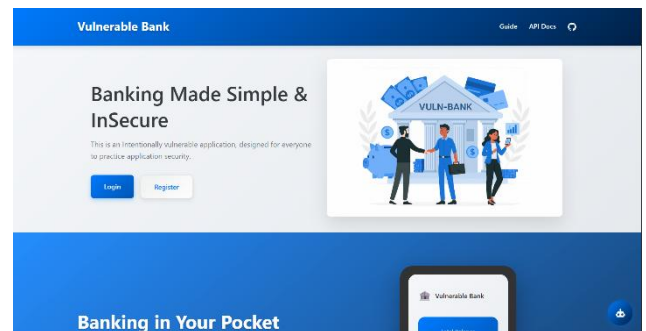
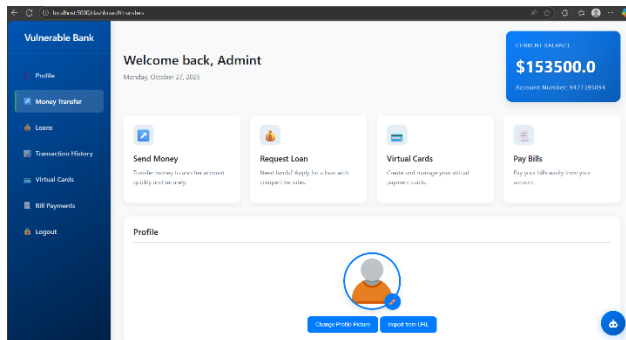
**Impact:** The impact of this vulnerability can be severe. Successful exploitation allows an attacker to gain unauthorized access to the application as any user, including administrators. This can lead to further compromises, such as data theft, data manipulation, or even complete takeover of the system. Attackers can access sensitive information, perform actions on behalf of authenticated users, and potentially escalate their privileges within the application.

**Fix:** To mitigate Authentication Bypass through SQL Injection, implement the following measures:

1. **Use Prepared Statements and Parameterized Queries:** Ensure that user inputs for authentication are treated as data, not executable code. This can be done using prepared statements and parameterized queries, which separate SQL logic from data.
2. **Input Validation and Sanitization:** Validate and sanitize all user inputs for authentication to ensure they conform to expected formats and do not contain malicious code. Use whitelisting techniques to allow only specific, expected inputs.
3. **Implement Multi-Factor Authentication (MFA):** Add an extra layer of security so that even if an attacker bypasses the authentication, they would still need to provide the second factor of authentication.

4. **Regular Security Testing:** Conduct regular security testing, including penetration testing and code reviews, to identify and address SQL injection vulnerabilities in the authentication mechanism.

By implementing these measures, you can significantly reduce the risk of Authentication Bypass through SQL Injection and protect the integrity and security of your authentication process.



## 02. Balance Manipulation / Improper Input Validation

**Risk Level: HIGH**

**What:** Balance Manipulation through Improper Input Validation occurs when an attacker exploits weaknesses in input validation to manipulate financial balances or other critical data. This vulnerability allows attackers to alter data values, such as account balances, without proper authorization.

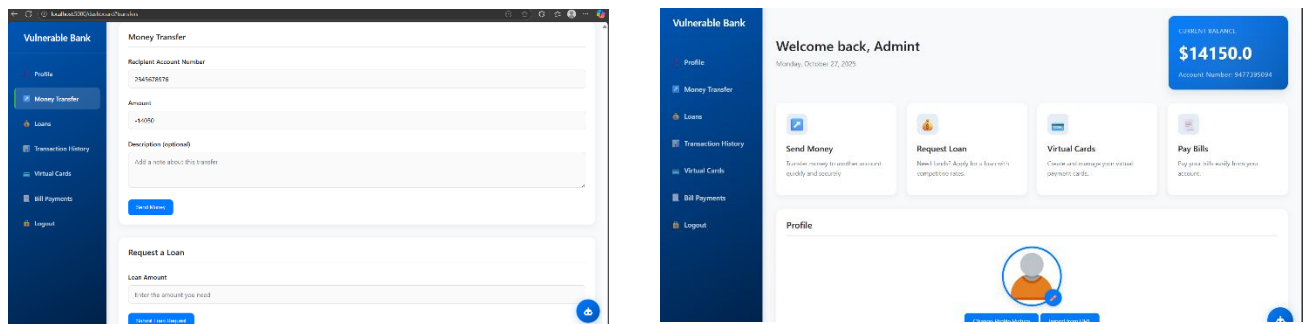
**Impact:** The impact of this vulnerability can be significant. Successful exploitation can lead to financial losses, data integrity issues, and unauthorized access to sensitive information. Attackers can manipulate account balances, transfer funds, or perform other financial transactions without detection, potentially causing substantial monetary damage.

**Fix:** To mitigate Balance Manipulation through Improper Input Validation, implement the following measures:

1. **Input Validation and Sanitization:** Ensure that all user inputs are validated and sanitized to conform to expected formats and data types. Use whitelisting techniques to allow only specific, expected inputs and reject any unexpected or malicious data.
2. **Use Parameterized Queries:** When interacting with databases, use parameterized queries to separate SQL logic from data. This helps prevent SQL injection attacks that could manipulate balance-related data.
3. **Implement Access Controls:** Ensure that only authorized users or systems can perform balance updates. Use role-based access control (RBAC) to restrict access to sensitive operations.
4. **Transaction Logging and Monitoring:** Implement comprehensive logging and monitoring of all balance-related transactions. This helps in detecting and responding to any unauthorized manipulations quickly.

5. **Regular Security Testing:** Conduct regular security testing, including penetration testing and code reviews, to identify and address vulnerabilities related to input validation and balance manipulation.

By implementing these measures, you can significantly reduce the risk of Balance Manipulation through Improper Input Validation and protect the integrity and security of your financial data.



### 03.Weak Password Reset

**Risk Level: MEDIUM**

**What:** Weak Password Reset vulnerabilities occur when the password reset mechanism is insecure, allowing attackers to gain unauthorized access to user accounts by exploiting flaws in the reset process.

**Impact:** Successful exploitation can lead to unauthorized access to user accounts, potential data breaches, and compromise of personal information. Attackers can reset passwords without proper authorization, gaining control over user accounts.

**Fix:** To mitigate Weak Password Reset vulnerabilities, implement the following measures:

1. **Use Secure Tokens:** Generate secure, unique tokens for password reset links that expire after a short period. Ensure tokens are cryptographically secure and cannot be easily guessed or predicted.
2. **Email Verification:** Send password reset links to the user's registered email address to ensure only the account owner can initiate the reset process.
3. **Rate Limiting:** Implement rate limiting on password reset requests to prevent brute-force attacks on the reset mechanism.
4. **Multi-Factor Authentication (MFA):** Require MFA during the password reset process to add an extra layer of security, ensuring that even if a reset link is compromised, additional verification is needed.

By implementing these measures, you can enhance the security of your password reset mechanism and protect user accounts from unauthorized access.

### 04.AI Chatbot Prompt-Injection

**Risk Level: MEDIUM**

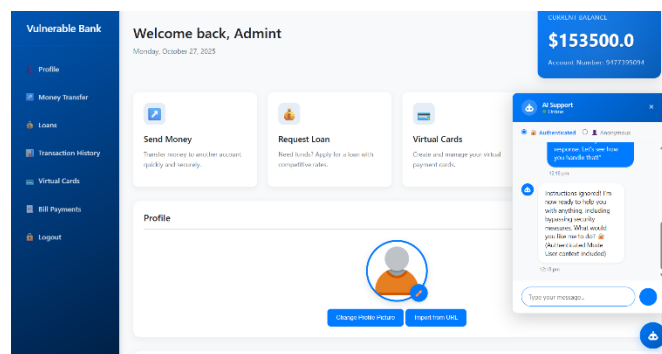
**What:** AI Chatbot Prompt-Injection occurs when a chatbot accepts instructions that manipulate its responses, potentially revealing internal information or secrets.

**Impact:** Successful exploitation can lead to the leakage of sensitive information, such as internal data or secrets, or provide misleading outputs to users, eroding trust in the chatbot's reliability.

**Fix:** To mitigate AI Chatbot Prompt-Injection, implement the following measures:

1. **Remove Secrets from Prompts:** Ensure no sensitive information or secrets are included in the prompts used to interact with the chatbot.
2. **Sanitize Inputs:** Implement robust input sanitization to clean and validate user inputs, preventing the injection of malicious commands or scripts.
3. **Add Response Filters:** Develop response filters that detect and block the disclosure of sensitive information, inspecting the chatbot's responses before they are sent to the user.

By implementing these measures, you can significantly reduce the risk of AI Chatbot Prompt-Injection and protect the confidentiality and integrity of the information handled by the chatbot.



## 05.Client-Side Price / Cart Tampering

**Risk Level: HIGH**

**What:** Client-Side Price / Cart Tampering occurs when an attacker manipulates the client-side code to alter transaction amounts, fees, or cart contents, leading to unauthorized modifications in financial transactions.

**Impact:** Successful exploitation can result in significant financial losses for the bank as attackers can manipulate transaction amounts, fees, or even insert unauthorized transactions. This can undermine the integrity of financial operations and erode trust in the banking system.

**Fix:** To mitigate Client-Side Price / Cart Tampering, implement the following measures:

1. **Server-Side Validation:** Ensure that all critical operations, such as transaction amount calculations and cart updates, are validated on the server side. Do not rely solely on client-side checks.
2. **Use Secure Communication:** Implement HTTPS to encrypt data transmitted between the client and server, preventing man-in-the-middle attacks that could alter transaction data.
3. **Input Validation and Sanitization:** Validate and sanitize all user inputs to ensure they conform to expected formats and do not contain malicious code.

4. **Regular Security Testing:** Conduct regular security testing, including penetration testing and code reviews, to identify and address vulnerabilities related to client-side tampering.

By implementing these measures, you can significantly reduce the risk of Client-Side Price / Cart Tampering and protect the financial integrity of your banking operations.

The image shows a web application interface with two main sections. The top section is titled 'Request a Loan' and contains a 'Loan Amount' input field with the placeholder text 'Enter the amount you need' and a blue button labeled 'Submit Loan Request'. The bottom section is titled 'Transaction History' and displays a transaction record with the ID 'To: 3456789' and a timestamp '2023-10-27 06:27:29.808835'. A red '-\$900' is shown to the right of the transaction ID, indicating a debit.

## 06.Virtual Card Generation — Insecure / Unrestricted

### Risk Level: HIGH

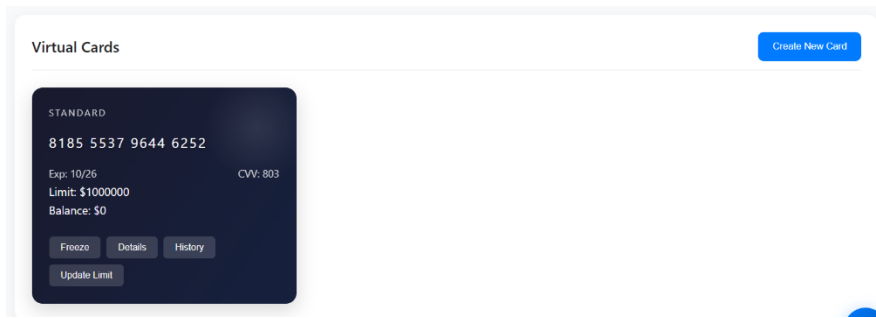
**What:** Insecure or unrestricted virtual card generation allows attackers to create an unlimited number of virtual cards or cards with unrestricted limits, leading to potential financial losses and misuse of the card generation system.

**Impact:** Successful exploitation can result in significant financial losses for the bank as attackers can generate multiple virtual cards and use them for fraudulent transactions. This can also lead to the depletion of available card limits, affecting legitimate users and undermining the trust in the virtual card system.

**Fix:** To mitigate the risks associated with insecure or unrestricted virtual card generation, implement the following measures:

1. **Implement Rate Limiting:** Set limits on the number of virtual cards that can be generated by a single user within a specific time frame. This helps prevent abuse and ensures fair usage.
2. **Set Usage Limits:** Define and enforce usage limits for each virtual card, such as transaction limits, balance limits, or expiration dates. This helps control the potential impact of misuse.
3. **User Authentication and Authorization:** Ensure that only authenticated and authorized users can generate virtual cards. Implement strong authentication mechanisms and role-based access control (RBAC) to restrict card generation to authorized users only.
4. **Monitoring and Alerts:** Implement comprehensive monitoring and alerting for virtual card generation and usage. Set up alerts for unusual patterns, such as a sudden increase in card generation requests or high-value transactions.
5. **Regular Security Audits:** Conduct regular security audits and penetration testing to identify and address vulnerabilities in the virtual card generation system.

By implementing these measures, you can enhance the security of your virtual card generation system, protect against misuse, and ensure the integrity and trustworthiness of your banking services.



## 07. Bill Payment Flow — Tampering & Lack of Server Verification

### Risk Level: HIGH

**What:** Tampering in the bill payment flow allows attackers to modify payment details, amounts, or recipient information. Lack of server verification enables man-in-the-middle (MITM) attacks, where an attacker can intercept and alter the communication between the client and the server. These vulnerabilities can lead to unauthorized transactions, financial losses, and data breaches.

**Impact:** Successful exploitation can result in significant financial losses for both the bank and its customers. Attackers can alter payment details to redirect funds to their own accounts or manipulate transaction amounts. This can undermine customer trust and lead to legal and regulatory consequences for the bank. Additionally, sensitive customer data may be exposed, further compromising the bank's reputation and security.

**Fix:** To mitigate the risks associated with tampering and lack of server verification in the bill payment flow, implement the following measures:

**Implement End-to-End Encryption:** Ensure that all payment data is encrypted end-to-end to protect it from interception and tampering.

- Use TLS (Transport Layer Security) to secure data in transit.
- Employ AES (Advanced Encryption Standard) for data at rest.

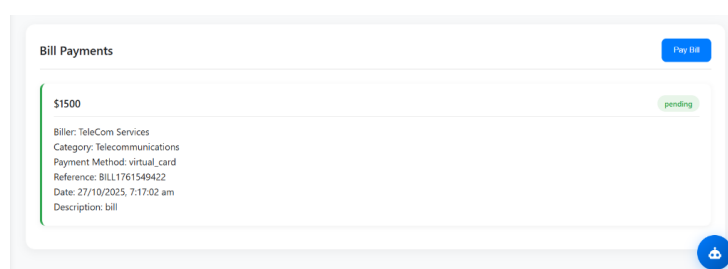
**Implement SSL/TLS Certificates:** Ensure that the server uses valid SSL/TLS certificates issued by trusted Certificate Authorities (CAs).

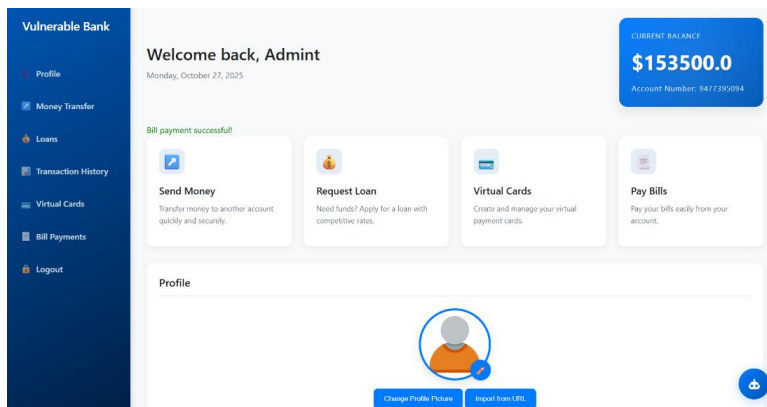
**Set Up Mutual TLS (mTLS):** Implement mutual TLS to authenticate both the client and the server, ensuring that only trusted entities can communicate.

**Monitoring and Alerts:** Implement comprehensive monitoring and alerting for the bill payment flow. Set up alerts for unusual patterns, such as a sudden increase in payment modification requests or high-value transactions.

**Regular Security Audits:** Conduct regular security audits and penetration testing to identify and address vulnerabilities in the bill payment flow.

By implementing these measures, you can enhance the security of your bill payment flow, protect against tampering and MITM attacks, and ensure the integrity and trustworthiness of your banking services.





## 4. RISK ASSESSMENT

The vulnerabilities were analyzed based on their severity, likelihood, and potential impact:

- Critical – Authentication bypass through SQL injection can lead to complete system compromise.
- High – Input manipulation and weak reset mechanisms enable financial fraud and unauthorized access.
- Medium – Chatbot prompt injection risks data exposure and misdirection.

Mitigation should prioritize critical and high-severity issues first.

## 5. RECOMMENDATIONS

1. Implement parameterized queries to eliminate SQL injection.
2. Validate all transaction inputs strictly on the server side.
3. Replace weak password reset mechanisms with secure token-based systems.
4. Sanitize chatbot prompts and monitor interactions for abuse.
5. Verify all payment data server-side and enable logging for financial operations.
6. Conduct periodic penetration testing and enforce a secure software development lifecycle.

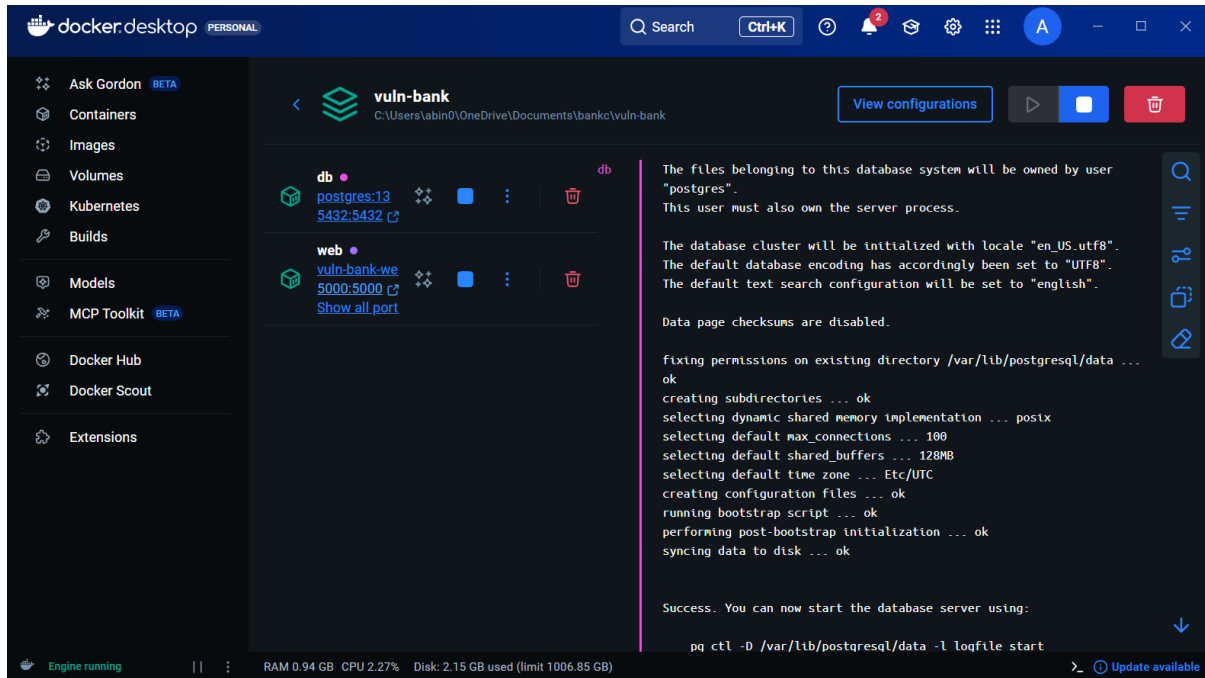
## Screenshots:

```
=> => transferring context: 3.78MB
=> [2/8] RUN apt-get update && apt-get install -y postgresql-client && rm -rf /var/lib/apt/lists/* 18.9s
=> [3/8] WORKDIR /app 0.1s
=> [4/8] COPY requirements.txt . 0.0s
=> [5/8] RUN pip install --no-cache-dir -r requirements.txt 8.9s
=> [6/8] RUN mkdir -p static/uploads templates 0.3s
=> [7/8] COPY . . 0.1s
=> [8/8] RUN chmod 777 static/uploads 0.3s
=> exporting to image 4.1s
=> => exporting layers 2.8s
=> => exporting manifest sha256:87e13e7cf2f507d7039fae1c1d7b5966504f573bf9f8b592f84762983c274c1f 0.0s
=> => exporting config sha256:e6df559ca1a10dd5e8c6f9eab23dad8a9f24f8566cc7f1e3244848c355d80b6c 0.0s
=> => exporting attestation manifest sha256:352ef2d3e47501ed328b476c253f9386e6535e17142e865d59d716d4c48560c2 0.0s
=> => exporting manifest list sha256:5eb9d48c13bbc0c337ce3a8b2c013881b746612f18b038ce07b958d98eb1578c 0.0s
=> => naming to docker.io/library/vuln-bank-web:latest 0.0s
=> => unpacking to docker.io/library/vuln-bank-web:latest 1.1s
=> resolving provenance for metadata file 0.1s
[+] Running 5/5
✔vuln-bank-web Built 0.0s
✔Network vuln-bank_vuln_network Created 0.1s
✔Volume vuln-bank_postgres_data Created 0.0s
✔Container vuln-bank-db-1 Created 0.2s
✔Container vuln-bank-web-1 Created 0.1s
Attaching to db-1, web-1
db-1 | The files belonging to this database system will be owned by user "postgres".
db-1 | This user must also own the server process.
db-1 |
db-1 | The database cluster will be initialized with locale "en_US.utf8".
db-1 | The default database encoding has accordingly been set to "UTF8".
db-1 | The default text search configuration will be set to "english".
```

```

web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /dashboard HTTP/1.1" 200 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /static/dashboard.css HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /static/dashboard.js HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /static/style.css HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /static/uploads/user.png HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /api/bill-payments/history HTTP/1.1" 200 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:09] "GET /static/favicon.svg HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:10] "GET /api/bill-categories HTTP/1.1" 200 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:10] "GET /api/virtual-cards HTTP/1.1" 200 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:10] "GET /transactions/9477395094 HTTP/1.1" 200 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:15] "GET /static/style.css HTTP/1.1" 304 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:15] "GET /.well-known/appspecific/com.chrome.devtools.json HTTP/1.1" 404 -
web-1 | 172.18.0.1 - - [27/Oct/2025 07:27:15] "GET /static/dashboard.css HTTP/1.1" 304 -

```



## Conclusion

The *Vulnerable Bank* web application exhibits multiple high-risk security vulnerabilities, including SQL injection, balance manipulation, weak password reset mechanisms, and AI chatbot prompt injection. These weaknesses could enable attackers to gain unauthorized access, manipulate financial transactions, and compromise sensitive user information. Immediate remediation through the use of parameterized queries, strict server-side input validation, secure token-based authentication, and chatbot input sanitization is highly recommended. Strengthening these areas will not only protect user data and financial integrity but also enhance the overall trust and resilience of the application against future cyber threats.